

A comparative study towards designing a hybrid architecture of microservices and LLM-based multi-agent systems

Yazdanian, P., Liu, Y., & Li, Z. (2026). A comparative study towards designing a hybrid architecture of microservices and LLM-based multi-agent systems. In *Proceedings of the 32nd Asia-Pacific Software Engineering Conference (APSEC 2025)* (pp. 761-772). (Proceedings of the Asia-Pacific Software Engineering Conference (APSEC)). IEEE. https://doi.org/10.1109/APSEC66846.2025.00077

Published in:
Proceedings of the 32nd Asia-Pacific Software Engineering Conference (APSEC 2025)

Document Version:
Peer reviewed version

Queen's University Belfast - Research Portal:
Link to publication record in Queen's University Belfast Research Portal

Publisher rights
Copyright 2025 the authors.

This is an accepted manuscript distributed under a Creative Commons Attribution License (https://creativecommons.org/licenses/by/4.0/), which permits unrestricted use, distribution and reproduction in any medium, provided the author and source are cited

General rights
Copyright for the publications made accessible via the Queen's University Belfast Research Portal is retained by the author(s) and / or other copyright owners and it is a condition of accessing these publications that users recognise and abide by the legal requirements associated with these rights.

Take down policy
The Research Portal is Queen's institutional repository that provides access to Queen's research output. Every effort has been made to ensure that content in the Research Portal does not infringe any person's rights, or applicable UK laws. If you discover content in the Research Portal that you believe breaches copyright or violates any law, please contact openaccess@qub.ac.uk.

Open Access
This research has been made openly available by Queen's academics and its Open Research team. We would love to hear how access to this research benefits you. – Share your feedback with us: http://go.qub.ac.uk/oa-feedback

Download date:03. Nov. 2026

A Comparative Study Towards Designing a Hybrid Architecture of Microservices and LLM-based Multi-Agent Systems

***Abstract*—LLM-based Multi-Agent Systems (LLM-MAS) present an emerging paradigm for constructing intelligent and adaptive applications that enable autonomous reasoning and collaborative problem-solving. Empirical studies show that current LLM-MAS still suffers from overlapping agent roles, unclear capabilities, and goal misalignment. In contrast, Microservice Systems (MS) are designed with modularity, and each service encapsulates a well-defined context and interface with structured and deterministic execution. From the system architecture principles, these paradigms demonstrate parallel attributes and complementary strengths that lead to a synthesized hybrid architecture. In this paper, we analyze the design factors of hybrid MS and LLM-MAS, as well as the main challenges, through a comparative study across eight architectural dimensions, including function encapsulation, orchestration, API design, autocorrection, data communication, operations, quality attributes, and environment awareness. The analysis reveals critical mismatches, design synergies, and transferable best practices. To motivate future work, we define four research questions to categorize the challenges. The goal is to create a converging design space for exploring architecture design towards intelligent, autonomous, modular, and adaptive systems.**

I. Introduction

An LLM-based agent is defined as "an artificial entity with prompt specifications (initial state), conversation trace (state) and ability to interact with the environment, such as tool usage (action)" [1]. LLM-based Multi-Agent Systems (MAS) enable the design of autonomous and collaborative problem-solving through independent reasoning and decision making. These agents interact through orchestration and achieve collective intelligence to solve complex tasks collectively [2]. LLM-MAS enables the system to adapt in different domains, resulting in a significant evolution in the field [1], [2], [3], [4]. Despite growing interest and attempt to adopt LLM-based MAS, an empirical analysis [1] has conducted an assessment over 200 tasks in leading frameworks such as ChatDev [5] and introduced the Multi-Agent System Failure Taxonomy (MAST). The taxonomy has identified 14 unique failure modes, categorized into three areas of poor system design, inter-agent misalignment, and task verification. The MAST taxonomy reveals that failures in LLM-MAS are primarily due to systematic design and coordination issues, rather than simply the limitation of individual LLMs or prompts. The analysis advocates fundamental and structural LM-MAS design to overcome the drawbacks of overlapping agent roles, unclear capabilities and goal misalignment.

The current works on LLM-MAS mainly focus on system design [6], [7], coordination [8], [9], communication and protocols [10], [11], [12], agent capabilities [13], [14], workflows [2], [15], frameworks [16], [17], [18] and inter-agent coordination misalignment [8], [9], [11], [15], [19].

Following well-defined modularity and boundaries with tool chains for deployment and operations [20], [21], microservice systems (MS) have deterministic execution paths with APIs for encapsulation, communication, and failover [20], [21]. Benchmarks of MS concentrate on assessing performance and scalability [22], comparing technology impact, establishing clear service boundaries, operational aspects such as observability and DevOps best practices, and Root Cause Analysis (RCA) [23], [24]. From the system architecture design, design paradigms in these two systems demonstrate parallel attributes. Hence, one conjecture of a new design initiative is to complement both strengths that lead to a synthesized hybrid architecture.

Motivation for the Hybrid Architecture: Both MS and LLM-MAS contain encapsulation of specific functionality and roles, but at different levels and approaches for coordination and orchestration. MS typically rely on centralized orchestration, and coordination is controlled externally outside of individual services, such as a central control point or service registries [20], [25]. Each microservice is designed to respond to API invocations reactively without an autonomous decision on execution [20], and the vision of the global objective is external to individual services. In contrast, MAS inherently offers decentralized orchestration using direct coordination among agents [8]. Each agent has the autonomy to decide on its participation, actions, and contribution towards achieving a high-level and global objective.

The hybrid architecture aims to bridge the orchestration paradigms in MS and LLM-MAS to allow an agentic system to act as an external context for managing microservices through autonomous communication among AI agents instead of static orchestration. Furthermore, the hybrid system can leverage the deterministic control of microservices to overcome issues of LLM-MAS (mentioned in MAST [1]) on agents' deviation from task specification, redundancy and repetition in operation, unawareness of conditions, as well as inter-agent misalignment. Finally, the hybrid architecture of MS and LLM-MAS aims to adapt to requirement changes and runtime conditions through reasoning rather than static logic modification.

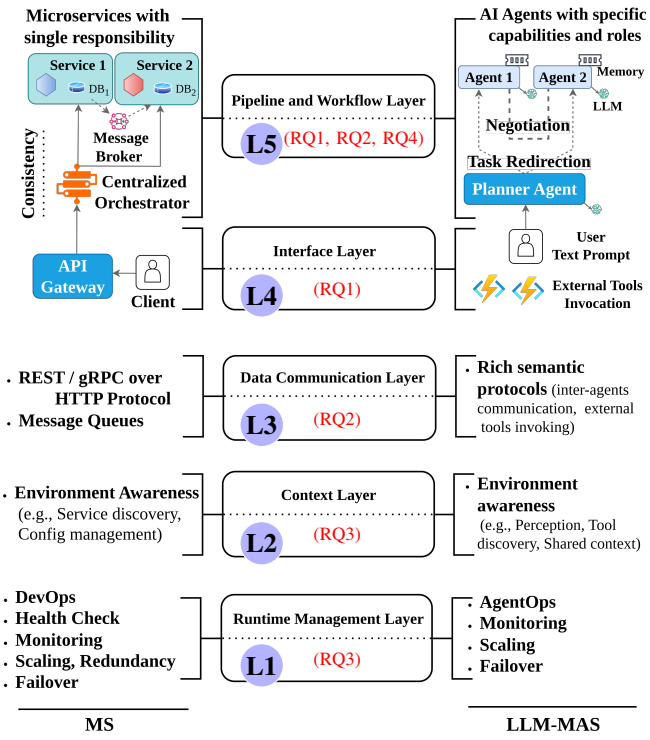


Fig. 1: A layered reference architecture for alignment of MS and LLM-MAS

In this paper, we analyze the design factors of hybrid MS and LLM-MAS to identify the main challenges through a comparative study. We outline a layered reference architecture that aligns both systems to derive eight architectural dimensions, including function encapsulation, orchestration, API design, auto-correction, data communication, operations, quality attributes, and environmental awareness. At each layer, we outline MS and LLM-MAS on two sides and annotate the key design factors. The analysis reveals critical mismatches, design synergies, and transferable best practices.

II. The Analysis Method

We develop a structured comparative method to study the integration of Microservice Systems (MS) and LLM-based Multi-Agent Systems (LLM-MAS). The objective is to identify transferable best practices and connect both paradigms through a hybrid architectural design. Unlike existing studies that primarily focus on system-level surveys or specific framework capabilities, our approach is grounded in a layered reference architecture, as depicted in Figure 1. The architecture organizes the system into coherent bottom-up functional blocks, including the runtime management, context, data communication, interface, pipeline and workflow. Each layer is used to align the functional roles and design assumptions of both systems. Based on this reference model, we outline the key design factors of MS and LLM-MAS on the two sides of each layer. The commonalities and mismatches in each layer are discussed in detail as follows.

A. *Microservice Systems*

*1) **Key Design Factors**:* The design of MS is of bounded context, which is the fundamental element of Domain-Driven Design (DDD) to enable the modular implementation of business logic in software [20]. It encapsulates each service with specific business functionality with a well-defined API and maintains its state and data using a dedicated database [20], while complex workflows are implemented by the coordination of individual services [20], [25]. The interface layer of microservices is mainly APIs based on a fixed schema to be consumed in public and internal communication through lightweight protocols such as REST or gRPC over HTTP [26]. Microservice systems might be deployed in various cloud or on-premise infrastructures and require awareness of the context and environment, such as external databases, profile (e.g., test, production), specific configuration [27], and dynamic discovery of other microservices for internal communication [20]. Another important factor in the design of microservices is independent scaling and redundancy to support failover, which is achieved through DevOps best practices and container orchestration tools [28].

*2) **Challenges**:* Despite its advantages and being widely adopted, MS presents several technical and organizational challenges. Most importantly, microservices lack reasonability and adaptivity due to implementation based on predefined logic. Furthermore, data inconsistency across multiple databases is a key challenge in MS that requires novel approaches to deal with (e.g., Saga pattern [29] for eventual consistency, two-phase commit to ensure strong consistency with atomic distributed transactions) [20]. Moreover, latency and disruption in the network necessitate resilience patterns for service communication in MS (e.g., circuit breakers, retries) [20], [30], [31]. Finally, MS require a complex lifecycle management for deployment, scaling, and observability based on a wide variety of tools and third-party services.

To address the reasonability and adaptivity problem of microservices, typical MS can be combined with Large Language Models (LLM) by invoking its main API through an external API proxy. This will support adaptation and provide cognitive capabilities for microservices, by reasoning using the LLM call during complex scenarios, to not rely solely on their static functionality [32]. However, this architecture is still limited to the direct reasoning of a single LLM. Such a system, although it increases adaptivity compared to typical MS, can not cope with dynamic behaviour and may not perform well even compared with a single-agent paradigm.

B. *LLM-Based Multi-Agent Systems*

*1) **Key Design Factors**:* Agentic AI systems that are augmented by LLMs provide an alternative framework to solve complex problems using collective intelligence [2], [1]. AI agents receive a text-based input prompt, reason autonomously based on their specific capabilities and roles through LLM, and maintain their own memory. Unlike microservices, coordination occurs here through collaboration between agents to share

beliefs and negotiate to solve conflicts in their planning using rich-semantic protocols in decentralized architecture [8], [33], or task delegation by a planner agent to specialized ones in a centralized architecture [8], [33]. Depending on how agent coordination is structured and the needed balance of autonomy with alignment, different architectural paradigms can be applied to MAS, such as centralized, decentralized (peer-to-peer), hierarchical, or hybrid topologies [2], [34]. Moreover, to interact with external systems, LLM-based agents invoke tools to perform operational activities for their decisions or fetch real-time data [2], [4], [8].

Since AI agents are stateful, contrary to microservices [35], the scaling of each agent is not achieved through pure replication. Therefore, supporting failover in LLM-MAS requires specific fallback mechanisms beyond infrastructure-based redundancy and health checks employed in MS. Moreover, AI agents need to perceive the environment to adapt their behaviour to dynamic changes [36], and discover available tools, based on their cognitive capabilities. In parallel to DevOps for MS, novel operational paradigms known as AgentOps [37] emerged in LLM-MAS to manage the agent's lifecycle and monitoring across all steps (e.g., LLM invocation, reasoning, collaboration, and external tool calling) [38].

*2) **Challenges**:* Designing robust and effective LLM-MAS at scale will raise its own technical challenges. Ensuring cohesion and avoiding inconsistency in the reasoning of agents are the most challenging aspects of LLM-MAS [8], [33]. Besides that, communication between agents can introduce latency and become bottlenecks at scale [10], due to the exchange of both data and intent. Last but not least, agents in LLM-MAS face stochastic behaviour in reasoning and decision-making, which makes reproducibility a key challenge [4].

III. The Taxonomy of the Comparative Study

This section introduces our comparative framework to analyze differences and commonalities between both systems, toward potential hybrid architectures. We identify eight critical architectural dimensions that define the core contrasts between the two paradigms. Through this dimensional analysis, we expose systematic mismatches, reveal design synergies, and extract transferable engineering practices such as fault isolation, continuous deployment, automated quality assurance, and security mechanisms.

A. Contrastive Analysis

Based on the assessment of MS and LLM-MAS from a multilayer perspective as in Figure 1, as well as key design factors and challenges of both systems analyzed in the previous section, they demonstrate fundamental differences in eight aspects, which are chosen to contrast them in our comparative study and discussed as follows. Table I presents the taxonomy we propose to systematically contrast MS and LLM-MAS in the defined set of dimensions, as well as the corresponding layer annotation shown in Figure 1.

*1) **Function Encapsulation**:* Microservices have clear and discrete boundaries that are human-defined using Domain Driven Development (DDD) [20]. This principle dictates the encapsulation of functionality in each service to allow it to contain all the logic and relevant data needed for a single responsibility [20]. Furthermore, this approach leads to the independence of microservices during deployment and scalability [20].

In contrast, function encapsulation is more dynamic and adaptive in LLM-MAS. It is embedded within the construction of each agent using prompts, and each agent encapsulates its functionality by defining the profile and skills, memory mechanisms, planning capabilities, and available tool set [35]. Furthermore, agent capabilities can be extended or refined based on evolving tasks, under-performance of another agent, or conversation context during runtime [39], which leads to fluid encapsulation. Unlike in MS where functional overlap is an anti-pattern, in MAS, the agent roles may be extended, and the boundaries between them become dynamic and not clear.

*2) **Orchestration**:* In MS, orchestration refers to the coordination of interactions between services to complete a flow in a centralized or decentralized manner [20], [25]. It is predefined as explicit workflows, and microservices do not decide to participate in a pipeline, as they should respond reactively to API invocations or events, and the vision of a global objective is external to them. In the centralized approach, an external service known as the orchestrator (e.g., Temporal, Camunda) manages the sequence of calls between microservices to complete a business flow [25]. Conversely, in decentralized choreography, microservices autonomously react to generated events in the system without a central control [25]. The challenge in centralized orchestration is the single point of failure of the central controller, while in the choreography pattern, the system may be chaotic as the number of microservices and events grows.

Orchestration within MAS is the process of coordinating multiple AI agents to achieve common goals. In contrast to MS, it is adaptive and autonomous in LLM-MAS, and AI agents make the decision about participating in the collaboration flow to achieve a global objective. There are various types of orchestration in MAS, such as centralized, decentralized, hierarchical, and federated [8], [40]. In the centralized approach, an LLM planner agent as a central controller organizes the activities of other specialized agents and delegates tasks to them [4], such as AutoGen [16] and MetaGPT [41]. Unlike in a decentralized orchestration, agents make independent decisions and reach a consensus as a group [4], [8], [40], such as AgentNet [42].

*3) **API Design**:* Since all data exchange between services occurs through messages and API calls, careful design of APIs as interfaces for microservices and versioning to manage evolution and backward compatibility is of great importance. These APIs may be exposed as explicit contracts (e.g. OpenAPI) for public clients, or Backend APIs to be used in interservice communication [20], [26]. API gateways play a

crucial role as an intermediary layer to manage all incoming requests and decouple clients from the direct invocation of microservices [20]. Another important aspect in API design is to ensure idempotent operations to obtain identical responses in multiple invocations without side effects [26]. Eventually, service invocation in MS is performed through well-defined contract-based APIs such as OpenAPI using REST or gRPC requests with predefined input/output schema [20], [26].

The concept of API design in LLM-MAS is different and primarily revolves around creating interfaces for interaction with external systems or APIs as tools. LLM agents leverage external tools to expand their capabilities and perform actions, such as accessing real-time data or manipulating databases [3]. This procedure involves understanding the tools available and the ability to determine whether a tool is necessary to solve a problem by agents [43], [44]. Hence, LLM-MAS require prompt-compatible API documentation including rich descriptions of purpose, schema, and example usage to enable accurate selection and invocation through reasoning in the absence of formal contracts. Moreover, the invocation of agents within LLM-MAS is based on prompting or high-level task delegation [35], which are often performed by agent orchestration frameworks such as AutoGen [16], Lang Graph [17] or Crew AI [18].

*4) **Auto-Correction**:* In MS, auto-correction and error handling for issues such as misinvocation or parameter missing are usually explicit and pre-programmed. Developers should manage failures and address such scenarios by ignoring and using a placeholder for missing data, throwing appropriate exceptions, or returning a defined Result Class object [45]. Otherwise, the system raises an error that may not be anticipated or handled by the clients.

LLM-MAS goes beyond legacy error handling by enabling agents to self-verify and self-correct their reasoning in the case of misinvocation or incorrect input. Frameworks such as S2.R [46] teach the LLM to constantly reassess its solutions and to be able to identify mistakes and self-check during the reasoning process. A novel approach used in InSeC [47], which applies self-correction directly into the LLM training process to obtain a more robust system.

*5) **Data Communication**:* Communication in MS is commonly syntactic, one-shot, and may be synchronous using RESTful/gRPC APIs over the HTTP protocol, or asynchronous event-driven architectures using message brokers [20], [48]. Communication protocols in MS are usually stateless and based on predefined schemas and interfaces, such as HTTP-based Representational State Transfer (REST) and Remote Procedure Call (RPC) [20]. These protocols are optimized for data exchange based on performance and schema rather than a semantic and intent-driven approach [48], [49].

Unlikely, agents' communication within MAS is inherently distributed and semantic-based for exchanging both data and intent, and usually multi-turn. Moreover, agents expose their capabilities or intentions, instead of static endpoints or interface contracts such as OpenAPI [48]. MAS communication protocols are stateful and based on rich-semantic and multi-turn negotiation, and are designed specifically for collaboration and belief/intention exchange between agents [48], [9], [50], or accessing external tools and data sources [12]. These protocols allow agents to negotiate, collaborate and adapt in dynamic environments, but introduce higher communication overhead and complexity [10]. Examples include legacy FIPA ACL (Agent Communication Language) and novel protocols [48], [50], such as MCP (Model Context Protocol) [12], [51], A2A (Agent-to-Agent protocol) [52], and ACP (Agent Communication Protocol) [53].

*6) **Environment Awareness**:* Microservices need to know the topology of the underlying infrastructure and dynamic variables of their environment for reliable communication or specific rules. For example, service discovery is a fundamental component, particularly in cloud environments, for loose coupling and seamless communication between microservices by abstracting the actual location of service instances [20]. Configuration management is another important aspect in awareness from different environments (e.g., development, test, production) in MS to relying on centralized repositories for dynamic configurations and variables [27]. Therefore, adaptation to changing environments in MS requires human intervention to update configurations or service discovery rules.

LLM agents are designed to perceive the environment and interpret input data in textual form and dynamically adapt their behaviour to changing conditions [36]. Furthermore, dynamic tool discovery and shared contexts between agents offered by new protocols such as MCP [12] are other key aspects of environment awareness in MAS. In contrast to MS, the awareness mechanism is more adaptive and related to the cognitive abilities of agents in MAS. This represents a paradigm shift from static mapping in MS to dynamic interpretation and discovery in environment awareness within LLM-MAS to support adaptivity.

*7) **DevOps vs. AgentOps**:* MS is deeply related to DevOps principles (e.g., continuous integration, automated testing, deployment and scaling, and extensive monitoring) to streamline software development, delivery, and maintainability [54]. In data-intensive or ML-powered microservices, MLOps extends these practices by training, versioning, and rolling out of machine learning or deep learning models, which are deployed and invoked within microservices. Observability is another key factor in DevOps to ensure service health and detect performance bottlenecks and functionality bugs at runtime in MS, which is achieved through standardized telemetry, log aggregation (e.g., ELK stack), cloud-based services (e.g., Datadog), and distributed tracing frameworks (e.g., OpenTelemetry, Zipkin) [20], [21], [54].

In contrast, LLM-MAS introduces novel operational paradigms known as AgentOps [37]. It extends the foundations of MLOps and LLMOps by introducing additional layers focusing on orchestrating agent lifecycles. The monitoring and observability mechanisms within AgentOps focus on tracking

LLM prompts and responses, reasoning and local beliefs, decisions, external tool calls, states and memories, and inter-agent interaction flows [37], [38], instead of hardware resource-based metrics or raw individual logs in DevOps. These mechanisms require different and novel tools (e.g., LangSmith, Langfuse). Guardrail is another concept of AgentOps that constraints and validates the input and output of LLM-MAS, to ensure that AI agents operate within defined boundaries and prevent them from taking unintended actions [55].

*8) **Quality Attributes**:* We consider redundancy and failover, scaling, consistency, CAP theorem tradeoffs, and evaluation metrics as different aspects under the quality attributes perspective for both MS and LLM-MAS.

Failover: In MS, several instances of a service are typically deployed to ensure redundancy, and the load is balanced between them using platforms such as Kubernetes. Upon detecting a failure in any instance (for example, through liveness or readiness probes in Kubernetes), automatic failover occurs by redirecting traffic to a healthy instance and removing the failed instance from the routing pool [28]. This mechanism is possible by being stateless of microservices, built-in support for redundancy in Kubernetes (e.g., replica-set and load balancer), and standardized health checks.

Unlikely, LLM-MAS face challenges in this approach, since AI agents are stateful and role-driven [35], making them harder to replicate purely. Furthermore, regular health check is replaced with runtime checks in agents for their reasoning and interactions [38], which is harder to observe and consider as a criterion for automatic failover. Hence, when an agent fails, whether due to poor reasoning, hallucination, or tool misuse, the system needs a fallback mechanism. Some architectural patterns, such as self-reflection loops or redundant fallback agents to reason independently on the same task, are discussed in [15] for soft failover. Overall, in contrast to MS, where failover relies on infrastructure redundancy (e.g., through simple replication, health check, and load balancing), failover in LLM-MAS requires high-level observability and fallback mechanisms and remains an open area for research.

Auto-scaling: In MS, the scaling unit is typically a container or pod, which is managed by container orchestration platforms such as Kubernetes. Auto-scaling is resource-driven and is achieved by monitoring utilization metrics such as CPU and memory usage, or request throughput constantly, which is managed by the Horizontal Pod Auto-scaler (HPA) Kubernetes feature [56]. Recent deep learning and graph-based approaches, such as DeepScaler [57] and Graph-PHPA [58], attempt to predict workload trend and proactively improve auto-scaling accuracy in MS under dynamic conditions.

In MAS, the scale unit is the agent or LLM-invoking component. Unlike MS, auto-scaling here is task-driven rather than resource-based, by triggers such as goal complexity, task queue depth, agent failure, or LLM invocation load. For example, a system may instantiate additional agents based on concurrent goal volume [59] or dynamically allocate more large language model instances if current LLM token queues overwhelm.

Consistency: The consistency in MS is mainly related to the data perspective (e.g. matching different versions of data in separate databases using strong or eventual consistency) [20]. Differently, consistency within MAS is a multifaceted challenge and is usually about synchronized shared beliefs and knowledge of agents, which are harder to achieve. Examples of consistency violations in MS are stale or conflicting data between services and separate databases, while in LLM-MAS, they appear as divergence of local beliefs among agents, objective misalignment, or uncoordinated action by a specific agent [60]. A novel solution to address these challenges employed in SagaLLM [61] by introducing ”transaction-like guarantees” and rollback mechanisms to ensure consistency and state recovery in multi-agent workflows, inspired by Saga pattern.

CAP Theorem Considerations: The CAP theorem states that a distributed system engages in a trade-off, in which it can only guarantee two aspects simultaneously, and should sacrifice one from consistency (C), availability (A), and partition tolerance (P) based on application needs [20]. In practice, MS typically prefer availability and partition tolerance (AP) for user satisfaction and accepts delayed consistency by employing techniques such as the Saga pattern, unless in applications in which data accuracy and strict consistency are critical, such as the financial domain [20].

However, in MAS, rather than achieving strong data consistency with synchronization, it is usually about local belief and state alignment among agents. The CAP theorem in its classic form (e.g. the triangle trade-off) may not be fully applied within MAS, and a weaker alternative such as LCAP (Local Consistency, Adaptability, Partition Tolerance) can better adapt to allow each agent to reason autonomously and eventually be adjusted via negotiation and belief sharing.

Quality Evaluation Metrics: In MS, quality metrics are focused on external performance and operational reliability (e.g. latency, throughput, performance, availability and error rates), often measured as service level indicators (SLIs) and can be combined under service level agreements (SLAs) [20].

Quality evaluations in MAS shift toward more enriched aspects (e.g., internal cognitive effectiveness and interaction performance) rather than static objectives. Since agents in a LLM-augmented MAS are goal-oriented, different metrics such as goal convergence, hallucination rate, reasoning convergence, and task and coordination success rate are more important to evaluate, rather than static KPIs [62].

B. Commonalities

Despite fundamental differences, MS and MAS share several core architectural principles. This section highlights the conceptual commonalities across both paradigms, which can be considered as the basis for the potential integration of microservices and AI agents. The proposed taxonomy for the commonality aspects between two systems is presented in table II and discussed as follows.

TABLE I: Hierarchical Contrastive Taxonomy between MS and LLM-MAS

Perspective	Aspect	MS Instance	LLM-MAS Instance
Function Encapsulation (L5)	Modularity Definition	Human defined	Dynamic and adaptive
		Contains own logic and data, Clear and discrete boundaries, Allow single responsibility [20]	Contains profile, skills and available tools [35], Allow extending and refining with unclear boundaries [39]
Orchestration (L5)	Coordination of Interactions	Predefined	Adaptive and autonomously
		Based on centralized orchestration and decentralized choreography with the right balance, Microservices are not aware of how they fit in a large system and must respond reactively to API invocations or events [20], [25]. Complete a flow with an external global objective for each service	Often is centralized (e.g., AutoGen [16], MetaGPT [41]) which coordinated by a central planner agent and task delegation to specialized agents, and decentralized (AgentNet [42]) where AI agents make decision to participate in the flow through collaboration to achieve a global objective [4], [8], [40]
API Design (L4)	Concept and Best Practices	Creating well-defined RESTful API Interfaces, exposed as public contracts or interservice, must consider versioning idempotent operations [26],	Creating interfaces for interaction with external tools, involves understanding the available tools and determining which ones are required [43], [44]
		Invocation through API gateway for centralized management for requests and decoupling client from microservices [20]	Invocation of individual agents happens through text prompt or high-level task delegation in frameworks
Auto-Correction (L4)	Handling misinvocation	Explicit and pre-programmed by developers, Implemented by legacy error handling [45]	Agents are able to self-verify and self-correct their reasoning from input using LLM [46], Self-correction can be directly embedded in LLM training process [47]
Data Communication (L3)	Mechanisms	Syntactic, One-shot, Synchronous or Asynchronous, Used for data exchange to invoke services or between them [20], [48]	Semantic based to exchange both data and intent, Asynchronous or Multi-turn, Used for collaboration and negotiation between agents, and external tool invocation [9], [12], [48]
	Protocols	Often stateless, Based on a rigid schema, Optimized for data exchange and performance [20], [48], [49] *e.g., REST call over HTTP to retrieve product list and price*	Stateful, Rich-semantic [9], [48], [50], May introduce higher complexity and overhead [10] *e.g., Negotiation via A2A protocol or External tool calling via MCP with messages containing purpose and data*
Environment Awareness (L2)	Focus and Approach	Concentrated on topology of the underlying infrastructure and reliable communication, Adaptation to new environments needs human intervention *e.g., Service discovery [20], Centralized configurations [27]*	Focused on interpretation from the environment, Achieved by cognitive abilities of agents, More adaptive [36] *e.g., Dynamic tool discovery and Shared context between agents in MCP [12]*
DevOps vs. AgentOps (L1)	Objective	Streamline software development, delivery, and maintainability	Orchestrating agent lifecycles including deployment and monitoring of AI Models and LLM based on MLOps/LLMOps
	Core Concepts	Continuous integration, Automated testing, Deployment, Scaling, Monitoring [54]	Deployment, versioning and monitoring of AI agents [37], Guardrails to ensure operation within defined boundaries [55]
	Observability Focus	Service health, Performance bottlenecks, Bugs in functionality of individual services, Distributed tracing across involved microservices [20], [21], [54]	LLM calls, agents planning and decisions, external tool calls, interaction between agents [37], [38], Lack of focus on hardware usage metrics and performance
Quality Attributes (L1, L5)	Failover (L1)	Achieved by infrastructure-based redundancy on microservices and failure detection [28]	Is not well defined in MAS due to stateful nature of agents, Each agent may be a single point of failure, Soft failover could be achieved using redundant fallback agents or self-reflection loops [15]
	Scaling Behaviour (L1)	Resource-driven (e.g., CPU or Memory usage, Average Requests)	Task-driven [59] (e.g., Goal complexity, Task queue, load of LLM calls)
		Individual microservices are scaled horizontally [56], Proactive approaches can be applied based on resource usage prediction [57], [58]	Additional AI Agents or LLM instances might be deployed
	Consistency (L5)	Synchronization of data across multiple databases with strong or eventual consistency [20] *e.g., Using Saga pattern instead of distributed transactions for eventual consistency across multiple databases*	Synchronized beliefs and alignment of objectives between agents [60] *e.g., SagaLLM [61] introduced "transaction-like guarantees" and rollback mechanisms for consistency and state recovery*
	CAP Theorem Considerations (L1, L5)	Consistency or availability might be scarified based on needs [20]	A weaker alternative with relaxed and local consistency is required to allow each agent reason autonomously and then be adjusted via belief sharing
	Evaluation (L1)	Focused on static service-level metrics [20], [63] *e.g., latency, availability, throughput, error rate, unavailability*	Measure cognitive and collaborative metrics [62] *e.g., Success rate in tasks and coordination, Belief drift rate, Dialogue turns needed for belief alignment*

*1) **Distribution and Autonomy**:* In MS, distributed microservices have their own databases and state, and coordinate with each other [64], [20] through centralized orchestration or decentralized choreography. Furthermore, each microservice encapsulates its own logic and lifecycle management to deploy and scale independently [20], [48]. In parallel, agents can make decisions autonomously in MAS and maintain their local state, memory, and individual objective, and collaborate toward a common goal [2], [4], [48].

*2) **Specialization of Components**:* In MS, each microservice implements a specific business functionality, following the single-responsibility principle, and is crucial for clear interfaces, maintainability. Closely, LLM-MAS include specialized agents with predefined roles and capabilities, each designed to solve a dedicated part of the overall workflow [2], [48].

*3) **Emergent Behaviour**:* Complex business logic is implemented within MS through the emergence of multiple local functionalities and interactions of loosely coupled microservices [20]. In parallel, collective intelligence in MAS is achieved by local reasoning and decision making of individual agents, and negotiation and cooperation between them [1], [4], [8], demonstrating an emergent behaviour. In both systems, the end-to-end functionality is not explicitly hard-coded, but rather is the result of the execution of several individual smaller functionalities and interactions between components.

C. Transferrable Best Practices from MS to MAS

Due to the same distributed and modular nature, several engineering best practices, which are widely adopted in MS can be meaningfully transferred to the design and implementation of LLM-MAS.

*1) **Separation of Concerns for Layered Architecture**:* In MS, modularity is achieved through the separation of different layers (e.g., user interface, business logic, and data persistence layers). Similarly, agents in MAS can follow this pattern to break up their end-to-end procedures into various layers, including data perception from the environment, reasoning/planning, acting, and memory management, to improve their maintainability. As shown in a recent survey on technical aspects of effective LLM-MAS, modular architectural strategies and layered infrastructure are emphasized as key points to build scalable and maintainable systems [6].

*2) **Resilience and Fault Isolation for Robust Execution**:* Fault tolerance and robustness are achieved in MS through patterns such as circuit breakers, retries with back-offs, and fallback mechanisms to stop cascading failure [20], [31]. Similarly, agents in a robust MAS should follow this mechanism to recover from action failures or plan inconsistencies, by techniques such as re-planning and delegating tasks to another backup agent. This contributes to the system's fault tolerance, preventing it from suffering a single point of failure.

*3) **Continuous Deployment with Quality Assurance Pipelines**:* A well-established set of unit and integration tests during continuous integration and deployment (CI/CD) pipelines is pivotal for quality assurance during roll-out of newer versions in MS. A similar mechanism should be extended to MAS, to simulate and validate individual performance and goal convergence of agents, as well as their interactions to collectively solve the problem in diverse scenarios, prior to deployment. An emerging best practice in [66] suggests evaluation-driven development in LLM-MAS, similar to test-driven development (TDD) in MS, where systematic benchmarks guide iterative refinement cycles.

*4) **Security Best Practices for Secure Agent Communication**:* Mechanisms such as authentication, authorization, and encryption are fundamental keys in securing MS [20], which should be applied in agent interactions in a secure MAS. For example, the identities of agents should be validated during invoking external critical resources (e.g., using OAuth or JWT). Furthermore, role-based access control (RBAC) as a well-adopted approach for hierarchical authorization in MS, can be used to regulate which service, tool, or other agent is authorized to be accessed by an agent within MAS.

Summary: Our comparative study demonstrates critical mismatches between MS and LLM-MAS in function encapsulation, orchestration, API design, data communication, operations (particularly in observability focus), environment awareness approach, and quality attributes such as failover, scaling, consistency.
Despite these differences in architectural dimensions, MS and LLM-MAS reveal design synergies such as distribution, autonomy, component specialization, and emergent behaviour in implementing complex workflows. Furthermore, the study identifies these four engineering best practices that can be borrowed from MS and applied directly to LLM-MAS: 1. Separation of concern by breaking-up procedures to various layers (e.g., reasoning, acting, memory management) 2. Resiliency and fault tolerance by employing fallback mechanisms such as re-planning and delegating tasks to a backup agent, 3. Developing CI/CD pipelines including automated test suite to validate individual and collective performance of agents before deployment, 4. Using security best practices in agents interaction (e.g., Authentication, Authorization, RBAC).

IV. Categorization of Open Research Questions

To motivate future work, we define four research questions to categorize the challenges to create a converging design space for exploring hybrid architecture design towards intelligent, autonomous, modular, and adaptive systems. These research questions focus on challenges within the integration of rigid and boundary-specific microservices with flexible and cognitive behaviour of AI agents, in the case of coordination, monitoring, quality assurance mechanisms, and enriching deterministic behaviour and explainability approaches for hybrid systems. Then, by analyzing the existing research on each challenge separately, we propose future research directions to address the main gap toward designing robust hybrid systems.

TABLE II: Commonalities Taxonomy between MS and LLM-MAS

Perspective	Shared Property in MS and MAS
Distribution and Autonomy (Layer 5)	Both are composed of loosely coupled, autonomous components (services or AI agents) that operate independently and coordinate at runtime [2], [20], [64], [65]. Components encapsulate internal state and logic, enabling independent operation and decision-making
	E.g., Microservices and agents independently perform tasks across distributed environments with loosely coupling
Component Specialization (Layer 5)	Both systems consist of role-specific units, each specialized in and responsible for a dedicated capability or functionality [2], [48], [65].
	E.g., authentication service in MS; price negotiation agent in MAS
Emergent Behaviour (Layer 5)	System-wide functionality arises from interactions between simple, modular units [4], [20].
	E.g., service orchestration pipelines in MS; collective problem solving in MAS

In Figure 1, each architectural layer and its related research questions are shown as **L** and **(RQ)** respectively.

Research Question 1

How are agents able to derive the knowledge of microservices for collaboration?

In a hybrid system, we face two main challenges in collaboration between agents and microservices, in L4 and L5 of the reference architecture shown in Figure 1:

Challenge 1) Microservice Semantic Comprehension: Effective collaboration between agents and services within a hybrid architecture requires the agent's ability to understand which microservices are registered, what functionalities they present, and how they should interact with them through individual API calls. This requires AI agents to mine the semantics of microservices to extract this knowledge in the form of metadata and how to invoke them (e.g., available service name and objectives, REST endpoints, input/output schema). The Model Context Protocol (MCP) as a middleware layer standardizes how AI agents access external applications and data sources by facilitating tool declaration and discovery using a specific schema [12]. However, it needs the exact schema and API definition for each external tool. Recent work performed in [67] demonstrates how LLMs can be used to extract high-level goals from the user prompt, decompose them into sub-goals, and then map them into actual API endpoints and automate invoking.

Future Research Directions: Although early efforts have been made to enable agents to mine microservice semantics in [67], and beyond MCP, there is a need for more formal methodologies and new tools to extract and represent microservice knowledge in a rich metadata format to be understandable to AI agents. In addition, maintaining this metadata is very important as microservices are constantly evolving. Therefore, a significant gap exists in how agents can dynamically discover new microservices or updated functionalities and changes in existing APIs or schema in real-time. Future research should concentrate on developing LLM agents capable of continuously monitoring the microservices environment (e.g., API gateway logs, telemetry data, OpenAPI contracts, schema registries). These agents should detect changes in the microservices API interface and update their internal knowledge representations such as OpenAPI specification or MCP tool descriptions.

Challenge 2) Inference of Implicit Service Dependencies: Beyond explicit API definitions for direct collaboration with each service, there are semantic dependencies between microservices themselves, demonstrated as hidden relationships or side effects between them. Hence, by calling a specific API from a service, it may be required to invoke some other dependent methods from the same or other microservices to fulfill a goal. An initial template to demonstrate how agents can identify the implicit relationships between microservices introduced in [68] by the concept of a Semantic Dependency Matrix, which proposes an automated approach to extract and represent these hidden relationships. Furthermore, to address similar behaviour in MAS, Causal Influence Diagrams are proposed in [69] to identify and mitigate decision risks and enable agents to anticipate harmful results, by providing a structured representation of cause-and-effect relationships.

Future Research Directions: To follow the continuous changes in microservices, the ability to constantly update a dynamic graph of dependencies between services is not well-researched. Furthermore, there is a lack of focus on the proactive behaviour of agents by inferring these dependencies and adjusting their plans to avoid conflicts in the orchestration of microservices. In designing LLM agents for a hybrid architecture, they should perform strict reasoning and verification of their action sequences to ensure that all side effects are considered based on the latest semantic dependencies of microservices.

Moreover, knowledge graphs are powerful mechanisms for capturing entities and their relationships and can provide a rich context for the LLM to perform retrieval-augmented generation (RAG) [70]. A similar idea can be applied to consider microservices as entities and capture their semantic dependencies between different APIs. This can improve the automation of calling microservices as external tools in the hybrid architecture coherently through the RAG mechanism empowered by knowledge graphs rather than a single LLM.

Research Question 2

How should agents resolve competition and conflict situations in coordination with a shared microservice?

Within a hybrid architecture, agents act as an orchestration layer that proxies decisions based on dynamic reasoning and delegates tasks to microservices, considering them as a worker role. The following problems are possible in the orchestration of microservices by AI agents:

Challenge 1) Agent Racing: Multiple agents can compete to invoke the same API of a common microservice, attempting to access or modify data simultaneously. This happens if agents identify the same microservice functionality as relevant to their goals based on their reasoning. Microservices themselves can use locking to ensure data consistency during concurrent access. Similarly, priority or scheduling rules define the order of resource access [8], and can be applied in L5 of the multilayered architecture in Figure 1, to avoid racing conditions for the same API invocation by multiple agents,

Future Research Directions: There is a critical research gap to deal with the agent racing problem on the agents' side in LLM-MAS using concurrency control mechanisms rather than relying on mechanisms such as locking in microservices. Hence, a possible direction for further research in hybrid systems can be dynamic prioritization and access management, which involves a meta-agent deciding to allocate access for a shared microservice to specialized agents. Moreover, introducing turn-based strategies and extending prioritization rules for external tool calling through agents' collaboration should be considered to deal with agent racing. Another approach may be the prediction of potential racing conditions based on agent plans and historical interaction patterns with microservices. This can enable the system to proactively prevent agent racing through the re-sequencing of tasks or temporarily block certain agent actions.

Challenge 2) Reasoning Inconsistency: Multiple agents that pursue their individual goals might reason to invoke a microservice, but with contrasting purposes. This leads to calling different APIs from a microservice with contradictory functionalities for the same data. In contrast to specific research on agent racing scenarios, which has not performed well, mechanisms for conflict resolution are extensively researched and can be directly applied in reasoning inconsistency situations. These mechanisms often involve negotiation between agents using iterative debates facilitated by novel protocols such as A2A [52] through L3 and L5 of the multilayered reference architecture in Figure 1, predefined rules [8], [15], [35], as well as auction-based or consensus-based mechanisms [4], [71]. Furthermore, using a third-party LLM to expand the knowledge boundary and in-depth debate proposed in [71], to enhance multi-agent consensus by analyzing uncertainty. In addition, [72] suggests that agents can adaptively bypass incorrect information and initial reasoning inconsistency, and eventually find alternative non-conflicting ways to achieve the overall objective, through adaptive and self-repairing capabilities.

Future Research Directions: Beyond consensus mechanisms and debate among agents for conflict resolution, a research gap lies in aligning the objectives of individual agents with system goals to prevent reasoning inconsistency. Future research should explore adaptive agent policies (e.g., using reinforcement learning with specific reward functions or a hierarchical planning structure) to ensure agents are continuously re-aligned with the global objective of the hybrid system. This approach can proactively avoid any contradiction between agents to invoke the microservices, rather than letting it happen and then relying on the mentioned conflict resolution mechanisms.

Research Question 3

How can observation and evaluation be performed within hybrid architectures?

There are two main challenges introduced within hybrid systems that are not an issue in MS or LLM-MAS.

Challenge 1) Requirement of Extensive Observability: Hybrid systems require double-sided observation of service-based aspects (e.g., latency, throughput, inter-service calls) and dynamic properties of agents (e.g., LLM calls, reasoning, belief/reasoning drift, tools invocation, goal convergence). Furthermore, monitoring in LLM-MAS is about understanding the evolving task and environment state, rather than metrics, and span L1 and L2 of the reference architecture depicted in Figure 1. Currently, both MS and LLM-based MAS contain their monitoring tools and cloud-based services (e.g., ELK stack, Prometheus, Datadog, OpenTelemetry for MS, and Langfuse, LangSmith, AgentOps-AI for LLM-MAS), each focused solely on service-based performance or agent-based dynamics. Therefore, extending current monitoring tools to trace audits of agents' planning, states, decisions, and all interaction between agents and services becomes essential.

Future Research Directions: Despite an enriched ecosystem for the monitoring of MS and LLM-MAS separately, there is a critical gap in the observability of hybrid systems. Further research and development should focus on monitoring the end-to-end cycle of request handling and attributing failures across agent–service boundaries. For example, tracking an audit of an input prompt spanning different agents, how they reason and collaborate, and what happens in the microservices thereafter, which are invoked as tools, using MS observation.

Challenge 2) Lack of Hybrid Evaluation Metrics: Standard MS metrics (e.g., latency, error rate, throughput) focus solely on performance-based aspects in execution and fail to capture emergent and dynamic behaviour. Conversely, LLM-MAS specific quality attributes (e.g., task completion, hallucination rate, goal convergence, belief drift, and inter-agent alignment) lack operational and performance metrics such as error budgets or latency. For example, works such as BattleAgentBench [62] and [73] provide early frameworks and quality assurance methodologies to evaluate agent reasoning, interactions, and capabilities, but focus merely on MAS

dynamics, lacking integration with service-oriented metrics. The benchmarks for hybrid systems must introduce new metrics measuring both operational robustness and cognitive performance (e.g., evaluating reasoning convergence under latency constraints, or distinguishing belief divergence rate from external error rate). The recent Survey on LLM-based agent evaluation [74] provides a structured taxonomy of evaluation methods appropriate for LLM-MAS; however, it notes the lack of a unified framework that combines both reasoning and performance indicators.

Future Research Directions: In parallel to the hybrid monitoring stack, the definition of integrated benchmark metrics and methodologies is required to merge the reliability and performance-based perspectives of microservices with the quality evaluation of MAS based on reasoning and coordination. Hence, in the hybrid system, the end user will be satisfied by an SLA covering not only its low latency and error rate but also its high-level, quantified adaptive reasoning and intelligence.

Research Question 4

How to achieve deterministic behaviour and explainability in a hybrid system?

Within a safety-critical business domain, dealing with the following challenges is of great importance:

Challenge 1) Indeterminism: Microservices are naturally designed to be deterministic in their responses through well-tested and hard-coded logic and execution path. Although factors such as distributed transactions, eventual consistency, and missed events in asynchronous communication usually disrupt this behaviour, several well-established techniques are introduced to deal with these problems and mitigate indeterminism in large-scale MS (e.g., idempotent operations in Saga pattern for eventual consistency [29], resiliency in communications with retry pattern [30]). In contrast, LLM-MAS suffer from extensive indeterminism primarily due to the probabilistic nature of LLM, as well as asynchronous dialogue-based interaction between agents, in L5 of Figure 1. Therefore, hybridization of both systems may lead to breaching deterministic behaviour as a fundamental aspect. Recent efforts have been made to improve the robustness of decisions and support determinism in LLM-MAS, such as ExACT [75], which proposes techniques such as the inference-time search algorithm, structured exploration and reflective planning.

Future Research Directions: Although absolute determinism may not be achieved in LLM-MAS, a research gap lies in defining bounded determinism. A possible direction of research in hybrid systems would be the development of techniques to formally specify the probabilistic behaviour of LLM agents and their impact on microservice interactions using acceptable ranges. This can provide guarantees even with inherent variability in the LLM; the hybrid system will remain within defined safety boundaries. Furthermore, similar to well-established techniques in MS to mitigate indeterminism, such as the Saga pattern, LLM agents should dynamically trigger compensatory actions to enforce consistency and determinism. Hence, future research should explore a control loop or a meta-agent which constantly monitors the real-time state of microservices and actions of agents. In case of any deviation from the deterministic behaviour, it should detect and trigger a set of compensation actions (e.g., re-plan agent actions, initiate recovery mechanism in microservices such as rollback or retry) to ensure consistency, which can directly lead to determinism.

Challenge 2) Lack of Explainability: Microservices are interpretable for both developers and non-technical stakeholders through architectural documentation, predefined execution paths, and best practices in coding. Although when a failure occurs, the attribution of the error and the analysis of the root cause are challenging in a large-scale MS. A large number of research studies have been performed for error attribution and root cause analysis (RCA) in microservices with various methods based on metrics, traces, multi-modal data, and LLM and AI agents [24].

In contrast, Artificial Intelligence (AI) is massively embedded in black-box models such as LLM, leading to growing concerns in LLM-MAS in the case of transparency and interpretability, particularly in safety-critical domains [76]. Similar to determinism, this transparency is required in the orchestration and function encapsulation of agents within the pipeline and workflow layer (L5) of the reference architecture in Figure 1. Recent works include automated failure attribution in LLM-based MAS conducted in [77], which supports explainability and accountability by identifying which agent caused the failure, under what conditions, and which reasoning trajectory. Another approach for applying XAI in MAS is layered prompting, which is performed in [76], by breaking down complex reasoning and decision-making into hierarchical and interpretable steps to improve transparency.

Future Research Directions: A significant gap is the lack of a unified explainability framework for the hybrid system, to provide explanations for decisions from a high-level text prompt through the agent's reasoning steps, its choice of microservices and the resulting interactions with their APIs. This would involve combining XAI techniques for LLMs with microservice observability of multi-modal data (e.g., logs, traces) to provide a complete chain of events in the hybrid system. Moreover, when a hybrid system fails, error attribution and root cause analysis between stochastic AI agents and deterministic microservice components are very challenging. Therefore, future endeavours should concentrate on extending current root cause analysis mechanisms for MS to cover agents' misreasoning and unexpected invocation of services.

V. Conclusion

Although Microservice Systems (MS) and Multi-Agent Systems (MAS) based on Large Language Models (LLM-MAS) are two main paradigms in the development of modern software applications, both suffer from their challenges and are not able to cope with the hybrid requirements of intelligent, modular, and adaptive systems. Furthermore, despite

having different roots and numerous contrasts, MS and LLM-MAS share important commonalities and transferable patterns, which can be considered as fundamentals toward designing a hybrid architecture to benefit the strengths of both paradigms, although this integration can introduce new challenges. Therefore, with the help of a comparative study between both MS and LLM-MAS, we derive four important research questions to identify the potential challenges and research gaps in each, and highlight the direction for future endeavours in designing robust, scalable, and trustworthy hybrid architecture.

REFERENCES

[1] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, "Why do multi-agent llm systems fail?" 2025. [Online]. Available: https://arxiv.org/abs/2503.13657

[2] X. Li, S. Wang, S. Zeng, Y. Wu, and Y. Yang, "A survey on llm-based multi-agent systems: Workflow, infrastructure, and challenges," *Vicinagearth*, vol. 1, no. 9, pp. 1–22, 2024. [Online]. Available: https://link.springer.com/article/10.1007/s44336-024-00009-2

[3] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large language model based multi-agents: A survey of progress and challenges," 2024. [Online]. Available: https://arxiv.org/abs/2402.01680

[4] N. Krishnan, "Ai agents: Evolution, architecture, and real-world applications," 2025. [Online]. Available: https://arxiv.org/abs/2503.12687

[5] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "Chatdev: Communicative agents for software development," 2024. [Online]. Available: https://arxiv.org/abs/2307.07924

[6] R. M. Aratchige and W. M. K. S. Ilmini, "Llms working in harmony: A survey on the technological aspects of building effective llm-based multi agent systems," 2025. [Online]. Available: https://arxiv.org/abs/2504.01963

[7] A. Fourney, G. Bansal, H. Mozannar, C. Tan, E. Salinas, Erkang, Zhu, F. Niedtner, G. Proebsting, G. Bassman, J. Gerrits, J. Alber, P. Chang, R. Loynd, R. West, V. Dibia, A. Awadallah, E. Kamar, R. Hosn, and S. Amershi, "Magentic-one: A generalist multi-agent system for solving complex tasks," 2024. [Online]. Available: https://arxiv.org/abs/2411.04468

[8] L. Sun, Y. Yang, Q. Duan, Y. Shi, C. Lyu, Y.-C. Chang, C.-T. Lin, and Y. Shen, "Multi-agent coordination across diverse applications: A survey," 2025. [Online]. Available: https://arxiv.org/abs/2502.14743

[9] X. Qiu, H. Wang, X. Tan, C. Qu, Y. Xiong, Y. Cheng, Y. Xu, W. Chu, and Y. Qi, "Towards collaborative intelligence: Propagating intentions and reasoning for multi-agent coordination with large language models," 2024. [Online]. Available: https://arxiv.org/abs/2407.12532

[10] G. Zhang, Y. Yue, Z. Li, S. Yun, G. Wan, K. Wang, D. Cheng, J. X. Yu, and T. Chen, "Cut the crap: An economical communication pipeline for llm-based multi-agent systems," 2024. [Online]. Available: https://arxiv.org/abs/2410.02506

[11] B. Yan, X. Zhang, L. Zhang, L. Zhang, Z. Zhou, D. Miao, and C. Li, "Beyond self-talk: A communication-centric survey of llm-based multi-agent systems," 2025. [Online]. Available: https://arxiv.org/abs/2502.14321

[12] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model context protocol (mcp): Landscape, security threats, and future research directions," 2025. [Online]. Available: https://arxiv.org/abs/2503.23278

[13] C. Zhang, K. Yang, S. Hu, Z. Wang, G. Li, Y. Sun, C. Zhang, Z. Zhang, A. Liu, S.-C. Zhu, X. Chang, J. Zhang, F. Yin, Y. Liang, and Y. Yang, "Proagent: Building proactive cooperative agents with large language models," 2024. [Online]. Available: https://arxiv.org/abs/2308.11339

[14] Q. Zhou, Z. Zhang, X. Xiang, K. Wang, Y. Wu, and Y. Li, "Enhancing the general agent capabilities of low-parameter llms through tuning and multi-branch reasoning," 2024. [Online]. Available: https://arxiv.org/abs/2403.19962

[15] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O'Sullivan, and H. D. Nguyen, "Multi-agent collaboration mechanisms: A survey of llms," 2025. [Online]. Available: https://arxiv.org/abs/2501.06322

[16] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "Autogen: Enabling next-gen llm applications via multi-agent conversation," 2023. [Online]. Available: https://arxiv.org/abs/2308.08155

[17] LangChain Developers, "Langgraph," 2025, part of LangChain Framework. [Online]. Available: https://www.langchain.com/langgraph/

[18] CrewAI Developers, "Crewai: The leading multi-agent platform," 2025, software Library. [Online]. Available: https://www.crewai.com/

[19] M. Choi, K. Kim, S. Chae, and S. Baek, "An empirical study of group conformity in multi-agent systems," 2025. [Online]. Available: https://arxiv.org/abs/2506.01332

[20] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. O'Reilly Media, 2021.

[21] "Microservices patterns." [Online]. Available: https://microservices.io/patterns/microservices.html

[22] C. M. Aderaldo, N. C. Mendonça, C. Pahl, and P. Jamshidi, "Benchmark requirements for microservices architecture research," in *Proceedings of the 1st International Workshop on Establishing the Community-Wide Infrastructure for Architecture-Based Software Engineering*, ser. ECASE '17. IEEE Press, 2017, p. 8–13.

[23] L. Pham, H. Zhang, H. Ha, F. Salim, and X. Zhang, "Rcaeval: A benchmark for root cause analysis of microservice systems with telemetry data," 2025. [Online]. Available: https://arxiv.org/abs/2412.17015

[24] T. Wang and G. Qi, "A comprehensive survey on root cause analysis in (micro) services: Methodologies, challenges, and trends," 2024. [Online]. Available: https://arxiv.org/abs/2408.00803

[25] Microsoft. Choreography pattern. Accessed: 2025-07-02. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/patterns/choreography

[26] Design apis for microservices. Accessed: 2025-07-02. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/microservices/design/api-design

[27] Moldstud, "Best practices for managing configuration in microservices: A comprehensive guide," *Moldstud Articles*. [Online]. Available: https://moldstud.com/articles/p-best-practices-for-managing-configuration-in-microservices-a-comprehensive-guide

[28] AWS Builders, "Ensuring high availability in microservices: A failover testing journey with aws," *dev.to*, 2023. [Online]. Available: https://dev.to/aws-builders/ensuring-high-availability-in-microservices-a-failover-testing-journey-with-aws-4aa5

[29] Microsoft. Saga pattern. Accessed: 2025-07-02. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/patterns/saga

[30] Retry pattern. Accessed: 2025-07-02. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/patterns/retry

[31] Application resiliency patterns. Accessed: 2025-07-02. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/architecture/cloud-native/application-resiliency-patterns

[32] R. Tarafdar, "Building Reliable LLM-Powered Microservices With Kubernetes on AWS," https://dzone.com/articles/reliable-llm-microservices-kubernetes-aws, May 2025, accessed: 2025-07-10.

[33] N. Krishnan, "Advancing multi-agent systems through model context protocol: Architecture, implementation, and applications," 2025. [Online]. Available: https://arxiv.org/abs/2504.21030

[34] T. Händler, "Balancing autonomy and alignment: A multi-dimensional taxonomy for autonomous llm-powered multi-agent architectures," 2023. [Online]. Available: https://arxiv.org/abs/2310.03659

[35] S. Han, Q. Zhang, Y. Yao, W. Jin, and Z. Xu, "Llm multi-agent systems: Challenges and open problems," 2025. [Online]. Available: https://arxiv.org/abs/2402.03578

[36] C. Jimenez-Romero, A. Yegenoglu, and C. Blum, "Multi-agent systems powered by large language models: Applications in swarm intelligence," 2025. [Online]. Available: https://arxiv.org/abs/2503.03800

[37] L. Dong, Q. Lu, and L. Zhu, "Agentops: Enabling observability of llm agents," 2024. [Online]. Available: https://arxiv.org/abs/2411.05285

[38] J. Kaur, "Agentops: The next evolution in ai lifecycle management," *XenonStack Blog*, May 2025. [Online]. Available: https://www.xenonstack.com/blog/agentops-ai

[39] O. Asici Tansu, Gurcan and G. Kardas, "Towards engineering llm-enhanced multi-agent systems: A critical examination of roles," 05 2025.

[40] M. Finio, "What is ai agent orchestration?" IBM Think Tech News, accessed Jun 2025. [Online]. Available: https://www.ibm.com/think/topics/ai-agent-orchestration

[41] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "Metagpt: Meta programming for a multi-agent collaborative framework," 2024. [Online]. Available: https://arxiv.org/abs/2308.00352

[42] Y. Yang, H. Chai, S. Shao, Y. Song, S. Qi, R. Rui, and W. Zhang, "Agentnet: Decentralized evolutionary coordination for llm-based multi-agent systems," 2025. [Online]. Available: https://arxiv.org/abs/2504.00587

[43] E. Lumer, A. Gulati, V. K. Subbiah, P. H. Basavaraju, and J. A. Burke, "Scalemcp: Dynamic and auto-synchronizing model context protocol tools for llm agents," 2025. [Online]. Available: https://arxiv.org/abs/2505.06416

[44] M. A. Haque, J. Williams, S. Siddique, M. H. Islam, H. Ali, K. D. Gupta, and R. George, "Advanced tool learning and selection system (atlass): A closed-loop framework using llm," 2025. [Online]. Available: https://arxiv.org/abs/2503.10071

[45] Enterprise Craftsmanship, "Error handling: Exception or result?" *Enterprise Craftsmanship Blog*. [Online]. Available: https://enterprisecraftsmanship.com/posts/error-handling-exception-or-result/

[46] R. Ma, P. Wang, C. Liu, X. Liu, J. Chen, B. Zhang, X. Zhou, N. Du, and J. Li, "S^2r: Teaching llms to self-verify and self-correct via reinforcement learning," 2025. [Online]. Available: https://arxiv.org/abs/2502.12853

[47] N. Upadhyaya and R. Sridharamurthy, "Internalized self-correction for large language models," 2024. [Online]. Available: https://arxiv.org/abs/2412.16653

[48] M. Goyal and P. Bhasin, "Moving from monolithic to microservices architecture for multi-agent systems," *World Journal of Advanced Engineering Technology and Sciences*, vol. 15, no. 1, p. 2119–2124, Apr. 2025. [Online]. Available: http://dx.doi.org/10.30574/wjaets.2025.15.1.0480

[49] M. Niswar, R. Safruddin, A. Bustamin, and I. Aswad, "Performance evaluation of microservices communication with rest, graphql, and grpc," *International Journal of Electronics and Telecommunications*, pp. 429–436, 06 2024.

[50] Y. Yang, H. Chai, Y. Song, S. Qi, M. Wen, N. Li, J. Liao, H. Hu, J. Lin, G. Chang, W. Liu, Y. Wen, Y. Yu, and W. Zhang, "A survey of ai agent protocols," 2025. [Online]. Available: https://arxiv.org/abs/2504.16736

[51] Anthropic, "Model context protocol (mcp)," Anthropic News, 2023, accessed: 2025-05-13. [Online]. Available: https://www.anthropic.com/news/model-context-protocol

[52] Google Developers, "A2a: A new era of agent interoperability," Google Developers Blog, 2024, accessed: 2025-05-13. [Online]. Available: https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/

[53] Agent Communication Protocol Developers, "Agent communication protocol: Introduction," Agent Communication Protocol Documentation, 2023, accessed: 2025-05-13. [Online]. Available: https://agentcommunicationprotocol.dev/introduction/welcome

[54] BDCC Global, "Devops and microservices: Best practices for scalable cloud applications," *BDCC Global Blog*. [Online]. Available: https://www.bdccglobal.com/blog/devops-microservices-best-practices-scalable-cloud/

[55] Infosys, "Agentops: Agentic lifecycle management for ai agents," *Infosys Knowledge Institute*. [Online]. Available: https://www.infosys.com/iki/research/agentops-agentic-lifecycle-management.html

[56] H. Ahmad, C. Treude, M. Wagner, and C. Szabo, "Towards resource-efficient reactive and proactive auto-scaling for microservice architectures," *Journal of Systems and Software*, vol. 225, p. 112390, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121225000585

[57] C. Meng, S. Song, H. Tong, M. Pan, and Y. Yu, "Deepscaler: Holistic autoscaling for microservices based on spatiotemporal gnn with adaptive graph learning," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '23. IEEE Press, 2024, p. 53–65. [Online]. Available: https://doi.org/10.1109/ASE56229.2023.00038

[58] H. X. Nguyen, S. Zhu, and M. Liu, "Graph-phpa: Graph-based proactive horizontal pod autoscaling for microservices using lstm-gnn," in *2022 IEEE 11th International Conference on Cloud Networking (CloudNet)*, 2022, pp. 237–241.

[59] Q. Wang, T. Wang, Z. Tang, Q. Li, N. Chen, J. Liang, and B. He, "Megaagent: A large-scale autonomous llm-based multi-agent system without predefined sops," 2025. [Online]. Available: https://arxiv.org/abs/2408.09955

[60] J. Hu, Y. Dong, S. Ao, Z. Li, B. Wang, L. Singh, G. Cheng, S. D. Ramchurn, and X. Huang, "Position: Towards a responsible llm-empowered multi-agent systems," 2025. [Online]. Available: https://arxiv.org/abs/2502.01714

[61] E. Y. Chang and L. Geng, "Sagallm: Context management, validation, and transaction guarantees for multi-agent llm planning," 2025. [Online]. Available: https://arxiv.org/abs/2503.11951

[62] W. Wang, D. Zhang, T. Feng, B. Wang, and J. Tang, "Battleagentbench: A benchmark for evaluating cooperation and competition capabilities of language models in multi-agent systems," 2024. [Online]. Available: https://arxiv.org/abs/2408.15971

[63] S. Li, H. Zhang, Z. Jia, C. Zhong, C. Zhang, Z. Shan, J. Shen, and M. A. Babar, "Understanding and addressing quality attributes of microservices architecture: A systematic literature review," *Information and Software Technology*, vol. 131, p. 106449, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950584920301993

[64] V. Bushong, A. S. Abdelfattah, A. A. Maruf, D. Das, A. Lehman, E. Jaroszewski, M. Coffey, T. Cerny, K. Frajtak, P. Tisnovsky, and M. Bures, "On microservice analysis and architecture evolution: A systematic mapping study," *Applied Sciences*, vol. 11, no. 17, 2021. [Online]. Available: https://www.mdpi.com/2076-3417/11/17/7856

[65] S. Chen, Y. Liu, W. Han, W. Zhang, and T. Liu, "A survey on llm-based multi-agent system: Recent advances and new frontiers in application," 2025. [Online]. Available: https://arxiv.org/abs/2412.17481

[66] B. Xia, Q. Lu, L. Zhu, Z. Xing, D. Zhao, and H. Zhang, "Evaluation-driven development of llm agents: A process model and reference architecture," 2025. [Online]. Available: https://arxiv.org/abs/2411.13768

[67] R. Feldt and R. Coppola, "Semantic api alignment: Linking high-level user goals to apis," 2024. [Online]. Available: https://arxiv.org/abs/2405.04236

[68] A. S. Abdelfattah, K. E. Cordes, A. Medina, and T. Cerny, "Semantic dependency in microservice architecture: A framework for definition and detection," 2025. [Online]. Available: https://arxiv.org/abs/2501.11787

[69] D. Hahm, W. Jin, J. S. Choi, S. Ahn, and K. Lee, "Enhancing llm agent safety via causal influence prompting," 2025. [Online]. Available: https://arxiv.org/abs/2507.00979

[70] X. Zhu, Y. Xie, Y. Liu, Y. Li, and W. Hu, "Knowledge graph-guided retrieval augmented generation," 2025. [Online]. Available: https://arxiv.org/abs/2502.06864

[71] Z. Duan and J. Wang, "Enhancing multi-agent consensus through third-party llm integration: Analyzing uncertainty and mitigating hallucinations in large language models," 2024. [Online]. Available: https://arxiv.org/abs/2411.16189

[72] T. Ju, B. Wang, H. Fei, M.-L. Lee, W. Hsu, Y. Li, Q. Wang, P. Cheng, Z. Wu, Z. Zhang, and G. Liu, "Investigating the adaptive robustness with knowledge conflicts in llm-based multi-agent systems," 2025. [Online]. Available: https://arxiv.org/abs/2502.15153

[73] I. Shamim and R. Singhal, "Methodology for quality assurance testing of llm-based multi-agent systems," in *Proceedings of the 4th International Conference on AI-ML Systems*, ser. AIMLSystems '24. New York, NY, USA: Association for Computing Machinery, 2025. [Online]. Available: https://doi.org/10.1145/3703412.3703439

[74] A. Yehudai, L. Eden, A. Li, G. Uziel, Y. Zhao, R. Bar-Haim, A. Cohan, and M. Shmueli-Scheuer, "Survey on evaluation of llm-based agents," 2025. [Online]. Available: https://arxiv.org/abs/2503.16416

[75] X. Yu, B. Peng, V. Vajipey, H. Cheng, M. Galley, J. Gao, and Z. Yu, "Exact: Teaching ai agents to explore with reflective-mcts and exploratory learning," 2025. [Online]. Available: https://arxiv.org/abs/2410.02052

[76] M. Sanwal, "Layered chain-of-thought prompting for multi-agent llm systems: A comprehensive approach to explainable large language models," 2025. [Online]. Available: https://arxiv.org/abs/2501.18645

[77] S. Zhang, M. Yin, J. Zhang, J. Liu, Z. Han, J. Zhang, B. Li, C. Wang, H. Wang, Y. Chen, and Q. Wu, "Which agent causes task failures and when? on automated failure attribution of llm multi-agent systems," 2025. [Online]. Available: https://arxiv.org/abs/2505.00212